



Lecture 3 : Transformation (génération de code) par template avec Acceleo

Ingénierie Dirigée par les Modèles (IDM / MDE) – version 2027

Du modèle Ecore `Library` au code Python, SQL et Java

Pr. Aissam BERRAHOU

Département Ingénierie des systèmes embarqués et mobiles – Smart Systems Lab
ENSIAS, Université Mohammed V de Rabat

aissam.berrahou@ensias.um5.ac.ma

Feuille de route

- **Pourquoi générer du code ?** : *Pourquoi écrire un programme qui écrit du code ?*
- **Premier projet Acceleo** : *Comment créer, écrire et lancer un premier générateur ?*
- **Le langage pas à pas** : *Quelles instructions pour parcourir le modèle : expressions, for, if, let, query, file ?*
- **Cas complet : Library.ecore** : *Comment générer Python, SQL et Java à partir de Library.ecore ?*
- **Bonnes pratiques et synthèse** : *Comment écrire un bon template ? Que retenir ?*

Objectifs de la séance

À la fin de cette séance, vous saurez :

- Expliquer ce qu'est une transformation **modèle** $\rightarrow$ **texte** (M2T) **par template** et sa place dans MDA.
- **Créer** un projet Acceleo dans Eclipse, **écrire** un module et le **lancer** sur un modèle.
- **Naviguer** dans un modèle Ecore avec des expressions OCL : `root.eClassifiers`, `cls.eAttributes`, `->filter(EClass)...`
- Utiliser les instructions `[for]`, `[if]`, `[let]`, `[file]`, `[query]`, `[protected]`.
- **Générer** à partir de `Library.ecore` des classes **Python**, un script **SQL** (clés primaires et étrangères) et des classes **Java** (constructeurs, getters/setters, `toString()`).

> **!** Lecture 1 (modèle, méta-modèle, Ecore), Lecture 2 (MDA, M2M avec ATL) et TP EMF (créer un `.ecore` avec Ecore Tools).

Pourquoi générer du code ?

Pourquoi écrire un programme qui écrit du code ?

Un constat : le code « mécanique »

→ Écrire 4 classes, c'est répéter 4 fois la même chose


```java
public class Book {
    private String title;
    public String getTitle() {
        return this.title;
    }
    public void setTitle(String title) {
        this.title = title;
    }
}
```

```java
public class Member {
    private String name;
    public String getName() {
        return this.name;
    }
    public void setName(String name) {
        this.name = name;
    }
}
```

- Les deux classes ont **exactement la même forme** : seuls les **noms** et les **types** changent.
- Ces noms et ces types sont **déjà écrits dans le modèle** `Library.ecore`. Pourquoi les retaper à la main ?

Ce que l'on gagne à générer

→ Quatre bonnes raisons

Productivité

On écrit le template **une fois** ; il produit le code de **toutes** les classes : 4, 40 ou 400.

Qualité et cohérence

Pas de faute de frappe ni d'oubli : toutes les classes suivent **les mêmes conventions**.

Plusieurs cibles

Le **même modèle** donne du Python, du SQL, du Java, du HTML… : un template par cible.

Suivre les évolutions

Le modèle change ? On **régénère**. Le code reste synchronisé avec le modèle.

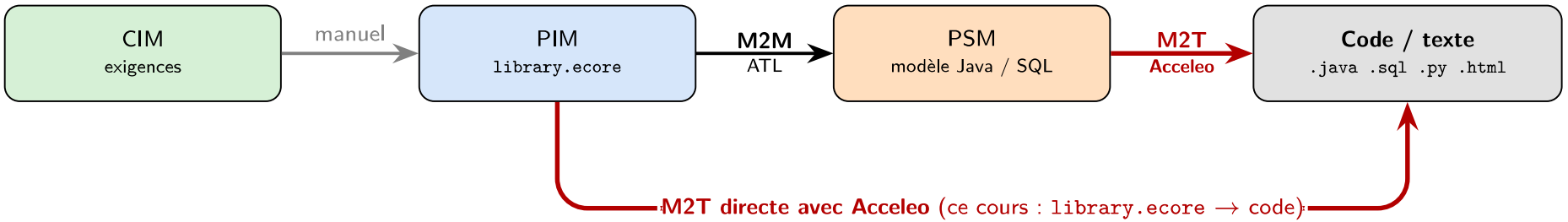
! En IDM, le **modèle** est la source de vérité ; le code en est **dérivé** automatiquement.

Où se situe Acceleo dans MDA ?

→ La dernière marche : du modèle au texte

CIM | PIM | PSM | **Code**

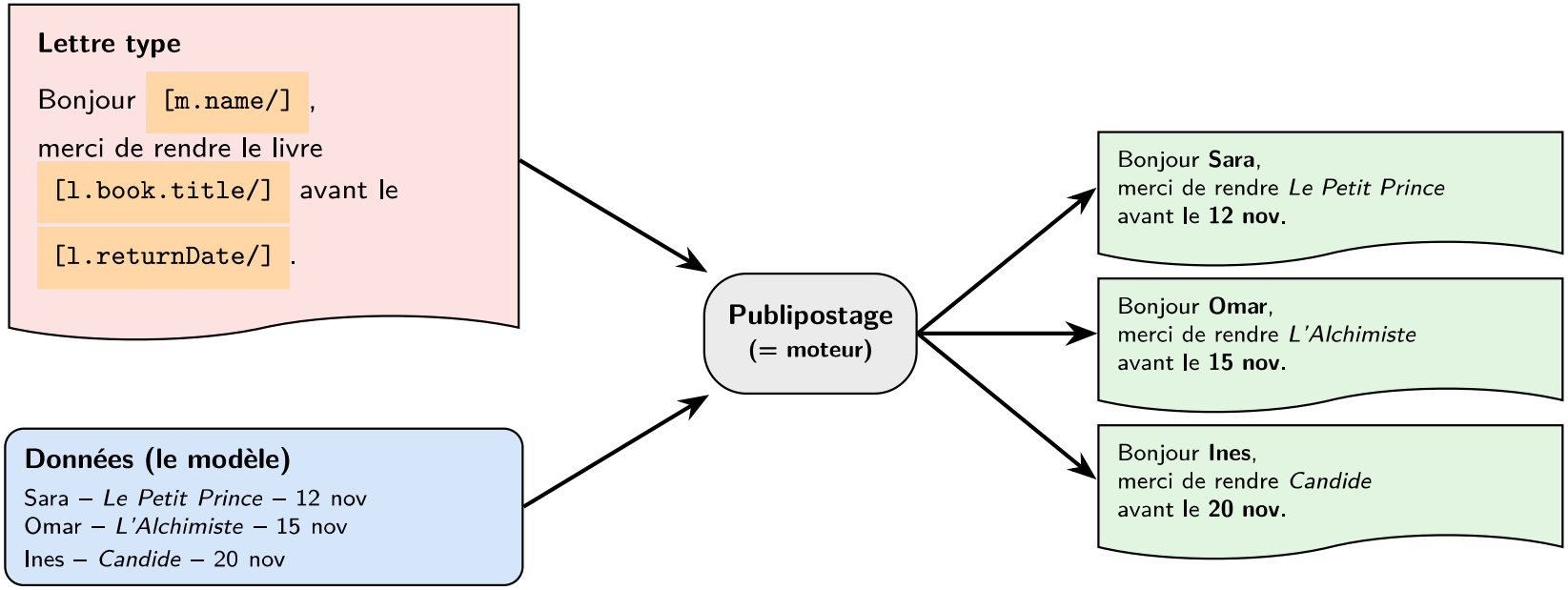
*Acceleo réalise la **dernière marche** de MDA : modèle → texte*



- **M2M** (Lecture 2, ATL) : un modèle → un autre **modèle**.
- **M2T** (aujourd'hui, Acceleo) : un modèle → du **texte** (code source, script SQL, HTML, configuration...).
- Dans ce cours, comme dans le tutoriel, on génère **directement** le code à partir du modèle `library.ecore`.

Une analogie : le publipostage

→ Une lettre type + une liste = autant de lettres que de lignes

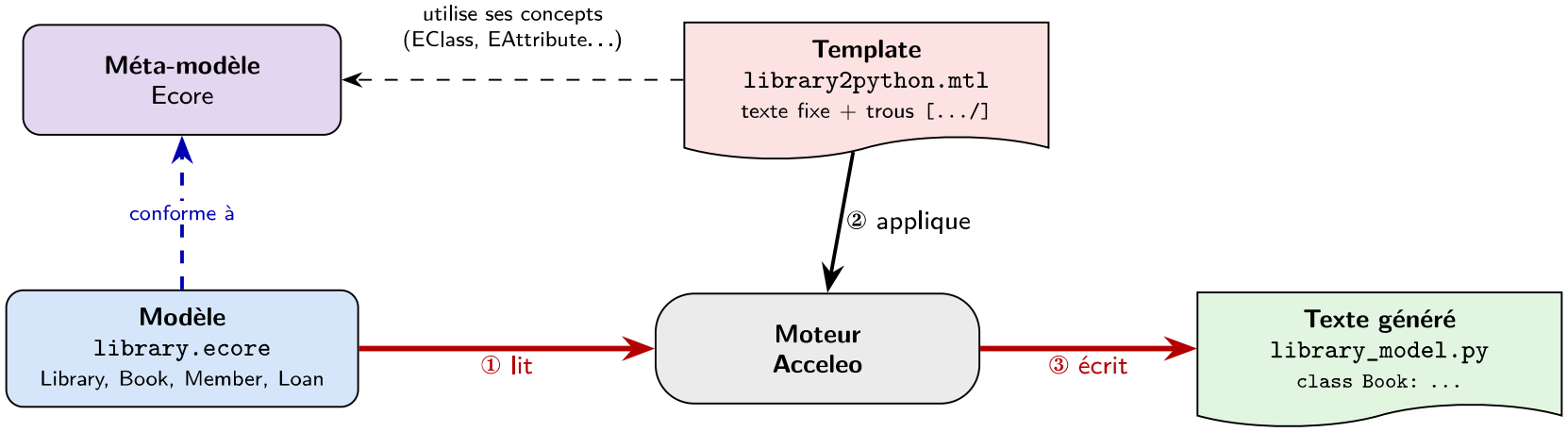


*Un **seul** texte à trous + des données = **autant** de textes que d'éléments. C'est exactement l'idée d'Acceleo.*

! Un **template** est un texte **à trous**. Le moteur remplit les trous avec les **données du modèle**.

Le principe de la génération par template

→ Trois ingrédients et un moteur



- **Modèle** : ce que l'on veut décrire (`library.ecore`).
- **Template** (`.mtl`) : le texte à produire, avec des **trous** `[ ... /]` qui lisent le modèle.
- **Moteur Acceleo** : il lit le modèle, applique le template et **écrit** les fichiers.

Acceleo en bref

- Générateur de code **open source** de la fondation **Eclipse** (créé par la société Obeo).
- Implémente le standard **MOFM2T** de l'**OMG** (« MOF Model to Text Transformation Language »).
- Lit tout modèle **EMF** (fichier `.ecore`, `.xmi`, `.uml`...) et produit **n'importe quel texte**.
- Les expressions s'écrivent en **OCL** (Acceleo 3) ou en **AQL** (Acceleo 4) : le même langage que pour les contraintes et ATL.

	Acceleo 3 (ce cours, le tutoriel)	**Acceleo 4 (version récente)**
Expressions	OCL	AQL (très proche)
Fichier	`[file (nom, false, 'UTF-8')]`	`[file (nom, overwrite, 'UTF-8')]`
Types	`EClass`, `EPackage`	`ecore::EClass`, `ecore::EPackage`

Pourquoi générer du code ?

→ Question flash

Question flash

?

Un collègue dit : « Un template Acceleo, c'est juste du code Java avec des `println` ».
Qu'est-ce qui rend l'approche par template **plus lisible** ?

Pourquoi générer du code ?

→ Question flash

Question flash

? Un collègue dit : « Un template Acceleo, c'est juste du code Java avec des `println` ». Qu'est-ce qui rend l'approche par template **plus lisible** ?

Réponse

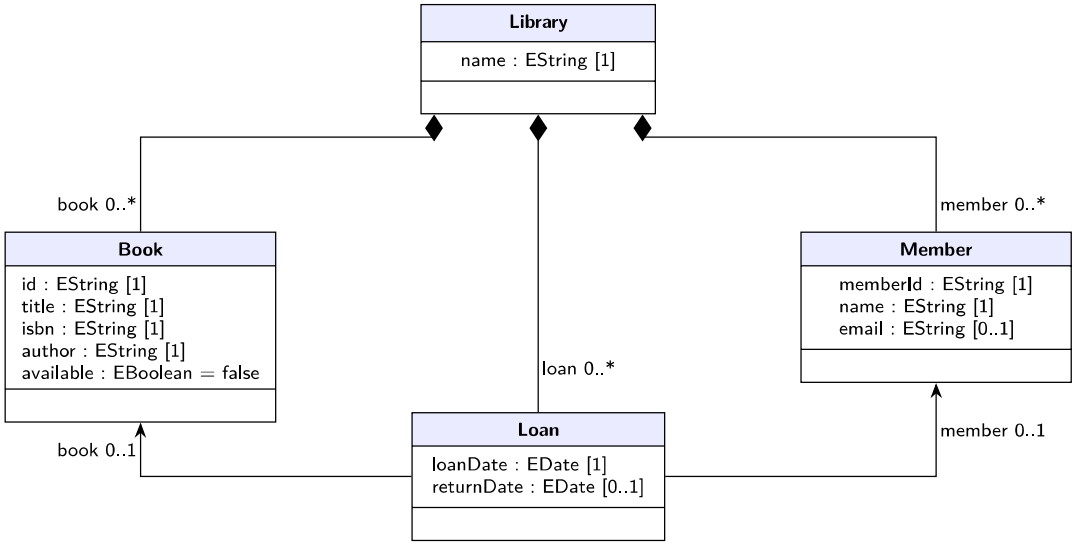
Dans un template, le **texte à produire est écrit tel quel**, en clair (on « voit » le fichier final) ; seules les **parties variables** sont entre crochets. Avec des `println`, c'est l'inverse : le texte est noyé dans des guillemets, des concaténations et des échappements.

Premier projet Acceleo

Comment créer, écrire et lancer un premier générateur ?

Le modèle d'entrée : Library.ecore

→ Le fil rouge du cours, du TD et du TP



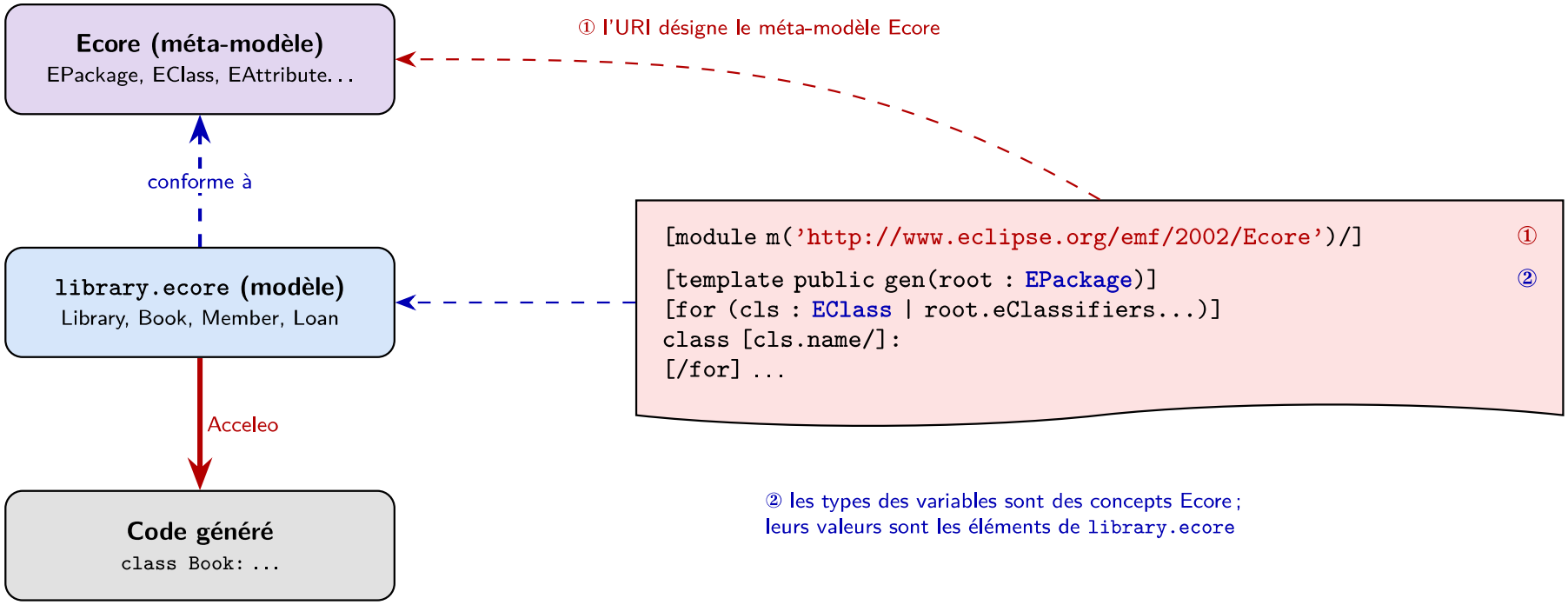
- **Library** : un nom ; contient des livres, des membres et des emprunts (**compositions** ⧫).
- **Book** : id, titre, ISBN, auteur, disponibilité (booléen).
- **Member** : identifiant, nom, e-mail (facultatif `[0..1]`).
- **Loan** : date d'emprunt, date de retour ; référence **un** livre et **un** membre (`0..1`).

> **!** Ici, le modèle d'entrée est **un fichier** `.ecore` créé avec Ecore Tools.

Le modèle est un .ecore : Acceleo « parle Ecore »

→ Les deux niveaux à bien distinguer

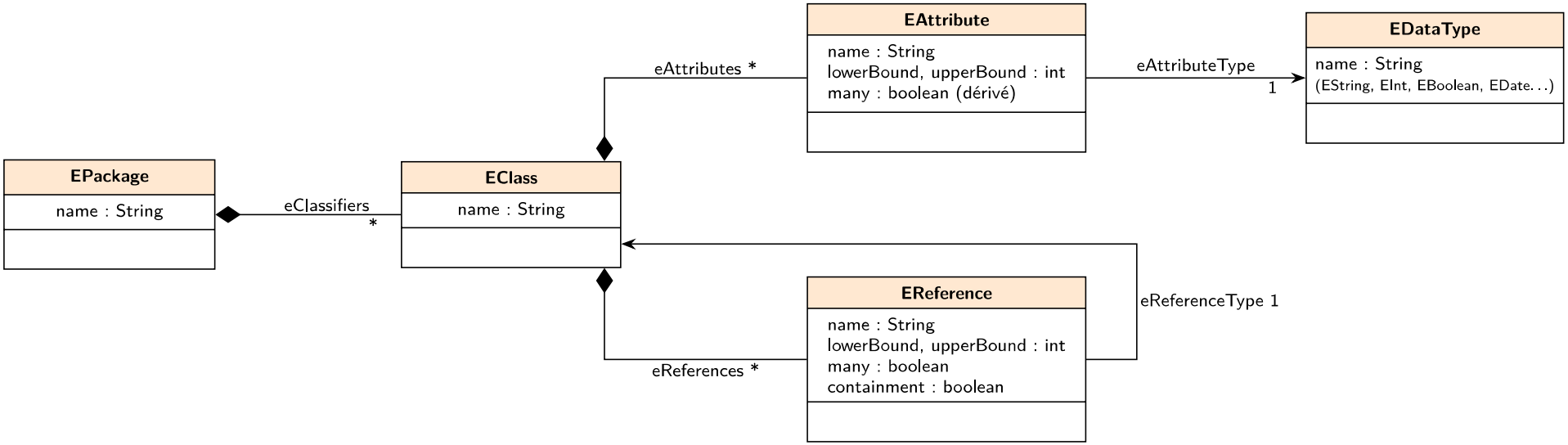
CIM | **PIM** | PSM | Code



*Ici le « modèle » d'entrée est lui-même un fichier `.ecore` : le template parle donc le langage **Ecore**.*
*Conséquence : un même template fonctionne pour **n'importe quel** fichier `.ecore`.*

Le méta-modèle Ecore (simplifié)

→ Les seuls concepts dont nos templates ont besoin

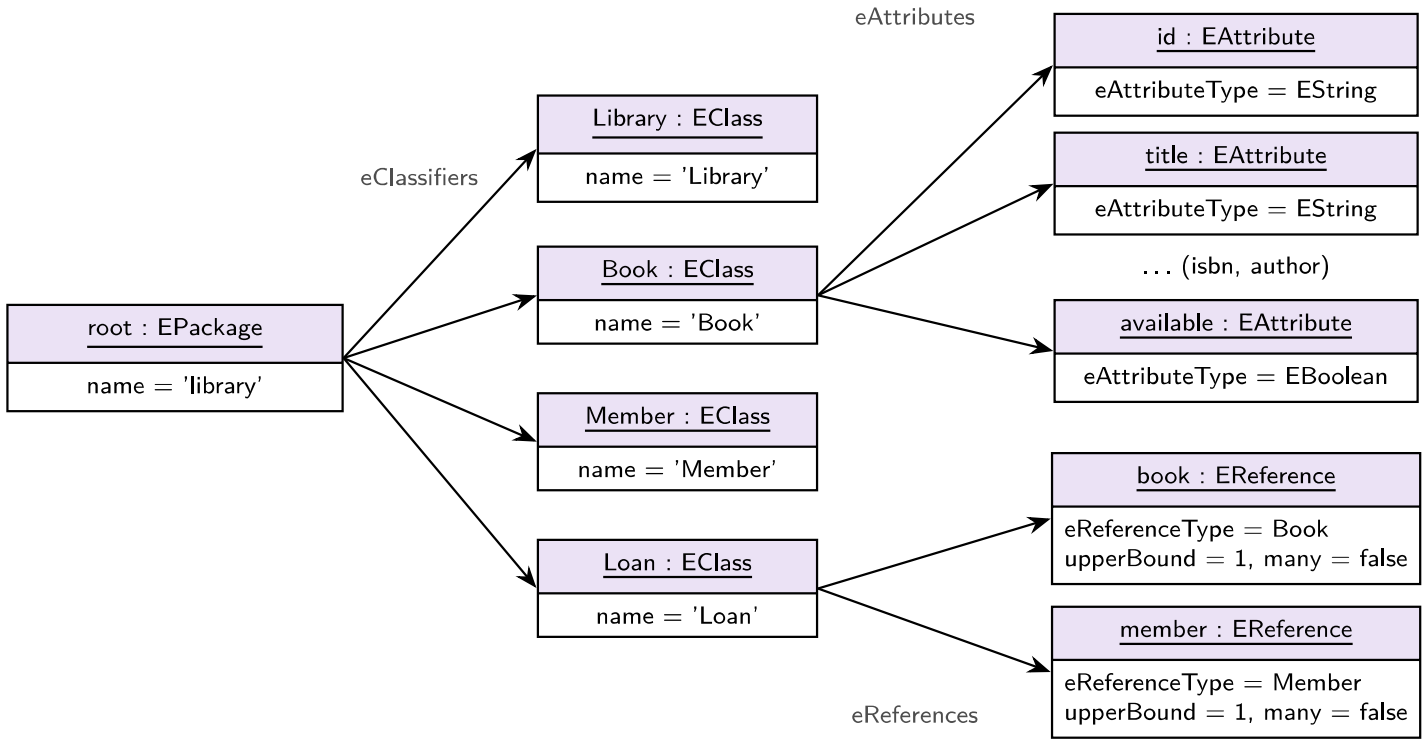


Ecore simplifié : les seuls concepts dont on a besoin pour écrire nos templates.
`eClassifiers` contient aussi des types de données : on filtre les classes avec `->filter(EClass)`.

- `many` vaut **vrai** si la borne supérieure est $*$ (ou > 1) ; `containment` = composition (⧫).

Ce que « voit » Acceleo

→ **library.ecore vu comme un graphe d'objets Ecore**



*Ce que « voit » Acceleo : un **graphe d'objets** Ecore. Le template **navigue** dans ce graphe en suivant les flèches.*

Installer Acceleo

→ Une seule fois, sur votre poste

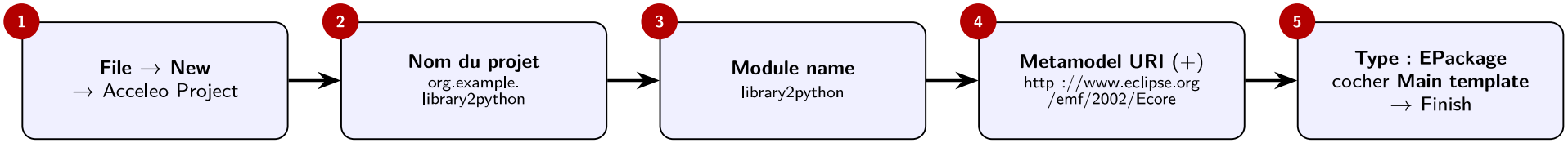
- Prérequis : **Eclipse Modeling Tools** (2024-xx ou plus récent) et un **JDK 17** ou supérieur.
- **Help** → **Install New Software...** ; dans « Work with », coller l'adresse du site de mise à jour :
  - Acceleo 3 (syntaxe de ce cours) : `https://download.eclipse.org/acceleo/updates/releases/3.7`
- Cocher **Acceleo**, puis **Next** → **Finish** ; accepter la licence et **redémarrer** Eclipse.
- Vérifier : le menu **File** → **New** → **Acceleo Project** existe (et la perspective « Acceleo » dans « Window → Perspective »).

Piège fréquent

Le site `.../releases/latest` installe **Acceleo 4** (syntaxe AQL, diapositive en fin de cours). Pour suivre le tutoriel et le TP **tels quels**, installez **Acceleo 3.7**.

Créer le projet Acceleo

→ L'assistant en 5 étapes

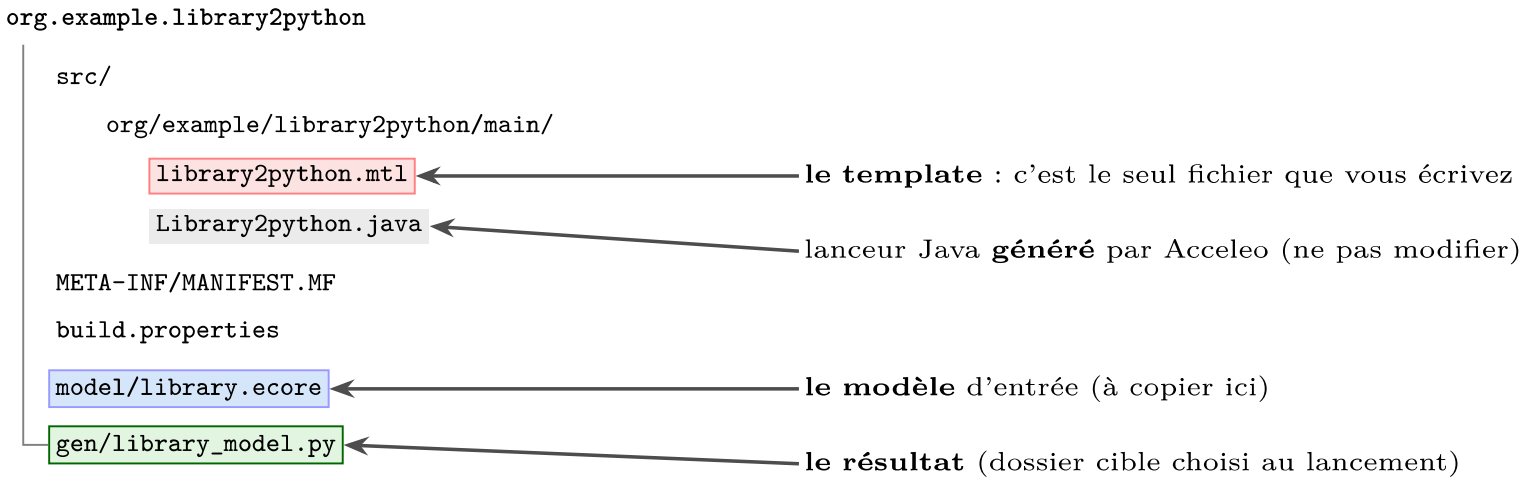


Résultat : un projet contenant `library2python.mtl` avec un template `generateElement(anEPackage : EPackage)` déjà marqué `@main`.

- L'URI `http://www.eclipse.org/emf/2002/Ecore` dit à Acceleo : « le modèle d'entrée est conforme au méta-modèle **Ecore** ».
- Le **type** choisi (`EPackage`) est le type de l'élément **racine** d'un fichier `.ecore`.

Structure du projet

→ Où sont les fichiers ?



- Copiez `library.ecore` dans un dossier `model/` du projet ; créez un dossier `gen/` pour les résultats.

Anatomie d'un module Acceleo

→ Le fichier .mtl ligne par ligne

```
[comment encoding = UTF-8 /]
[module library2python('http://www.eclipse.org/emf/2002/Ecore')/]

[template public generateElement(root : EPackage)]
[comment @main/]
[file ('library_model.py', false, 'UTF-8')]
# Code Python généré automatiquement
[for (cls : EClass | root.eClassifiers->filter(EClass))]
class [cls.name/] :
[/for]
[/file]
[/template]
```

1. Encodage du fichier .mtl
2. **Module** = nom du fichier + **URI du méta-modèle**
3. **Template** : paramètre typé (`EPackage`)
4. **@main** : point d'entrée (comme `main()`)
5. **File** : crée le fichier de sortie
6. **Texte statique** : recopié tel quel
7. **For** : une itération par classe
8. **Expression** `[.../]` : remplacée par sa valeur
9. Chaque bloc ouvert est **refermé** `[/...]`

Code couleur : **violet** = déclarations, **bleu** = balises de bloc, **gris** = texte statique, **orange** = expression.

Étape 1 : le point d'entrée

→ Un fichier de sortie avec un en-tête fixe


```
[comment encoding = UTF-8 /]
[module library2python('http://www.eclipse.org/emf/2002/Ecore')/]

[template public generateElement(root : EPackage)]
[comment @main/]
[file ('library_model.py', false, 'UTF-8')]
# -*- coding: utf-8 -*-
# Code Python généré automatiquement à partir du modèle Ecore
[/file]
[/template]
```

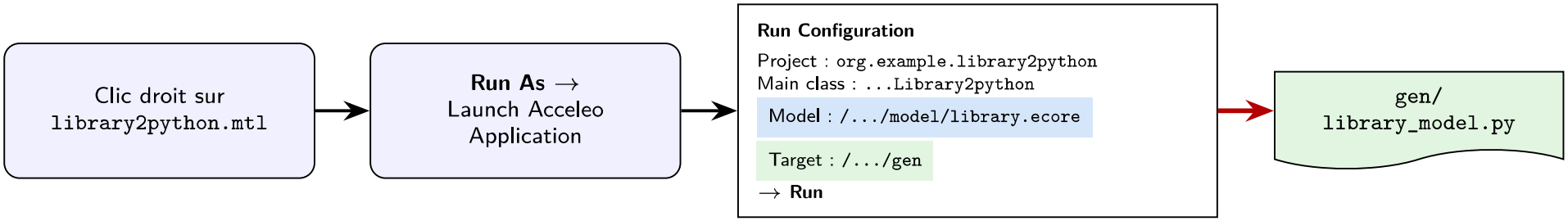
Résultat : gen/library_model.py

```
# -*- coding: utf-8 -*-
# Code Python généré automatiquement à partir
      du modèle Ecore
```

- Rien n'est encore lu dans le modèle : tout est **statique**.
- `false` = **écraser** le fichier s'il existe (`true` = ajouter à la fin).

Lancer la génération

→ Run As → Launch Acceleo Application



*Astuce : faites **F5** (Refresh) sur le projet pour voir le dossier gen/. Ensuite, un simple clic sur **Run** relance la génération.*

- **Model** : le fichier d'entrée (`library.ecore`). **Target** : le dossier où écrire (`gen`).
- La configuration est mémorisée : on relance ensuite d'un clic sur **Run**.

Étape 2 : une classe Python par EClass

→ Première boucle sur le modèle


```
[file ('library_model.py', false, 'UTF-8')]
# -*- coding: utf-8 -*-
[for (cls : EClass | root.eClassifiers->filter(EClass))]

class [cls.name/]:
    pass
[/for]
[/file]
```

- `root.eClassifiers` : toutes les classes **et** types du paquetage.
- `->filter(EClass)` : ne garder que les **EClass**.
- `[cls.name/]` : le **nom** de la classe courante.

Résultat

```
# -*- coding: utf-8 -*-

class Library:
    pass

class Book:
    pass

class Member:
    pass

class Loan:
    pass
```

Question flash

?

On ajoute au modèle une 5e classe `Author`, puis on relance **Run**. Que faut-il modifier dans le template pour obtenir `class Author:` ?

Question flash

? On ajoute au modèle une 5e classe `Author`, puis on relance **Run**. Que faut-il modifier dans le template pour obtenir `class Author:` ?

Réponse

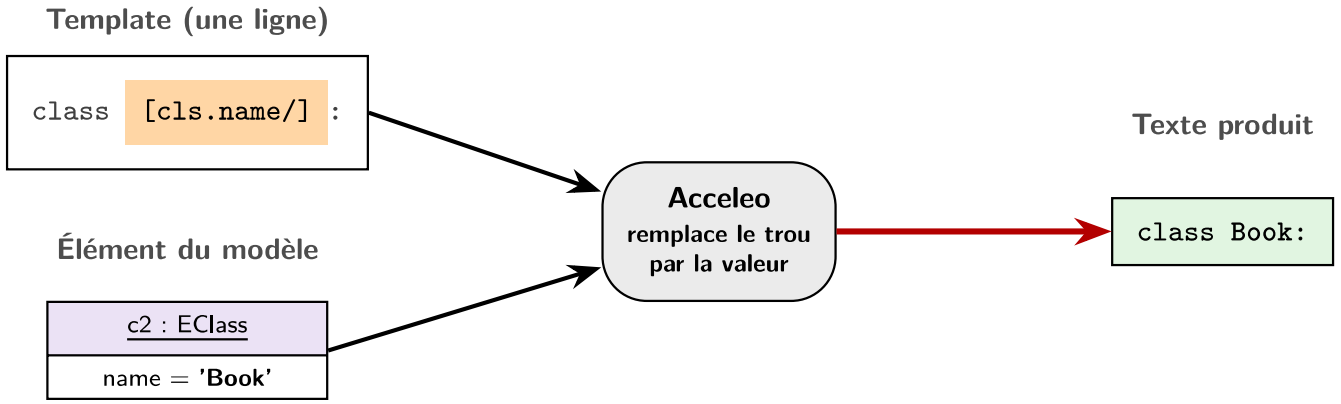
Rien du tout ! Le template parcourt `root.eClassifiers` : la nouvelle classe est trouvée automatiquement. C'est tout l'intérêt de la génération : on change le **modèle**, pas le générateur.

Le langage pas à pas

Quelles instructions pour parcourir le modèle ?

Texte statique et expressions

→ Tout ce qui n'est pas entre crochets est recopié



*Gris = **texte statique** recopié tel quel | orange = **expression** [. . ./] évaluée sur le modèle*

- `[expression/]` : **évalue** l'expression sur le modèle et écrit le résultat (le `/` ferme l'expression).
- Pour écrire un **crochet** dans le résultat, on écrit une chaîne : `['['/]` et `[']'/]` (ex. une liste Python vide).

Naviguer dans le modèle avec OCL

→ Le point suit une flèche du méta-modèle



*Chaque **point** = suivre une flèche du méta-modèle.* `attr.eAttributeType.name` → `'EBoolean'`

Expression	Signification	Valeur sur Library.ecore
`root.name`	nom du paquetage	'library'
`root.eClassifiers->filter(EClass)`	les classes du paquetage	Library, Book, Member, Loan
`cls.eAttributes`	les attributs d'une classe	Book : id, title, isbn, author, available
`cls.eReferences`	les références (associations)	Loan : book, member
`attr.eAttributeType.name`	nom du type Ecore	'EString', 'EBoolean', 'EDate'
`ref.eReferenceType.name`	nom de la classe cible	Loan.book → 'Book'

Les opérations OCL les plus utiles

→ La flèche -> s'applique à une collection

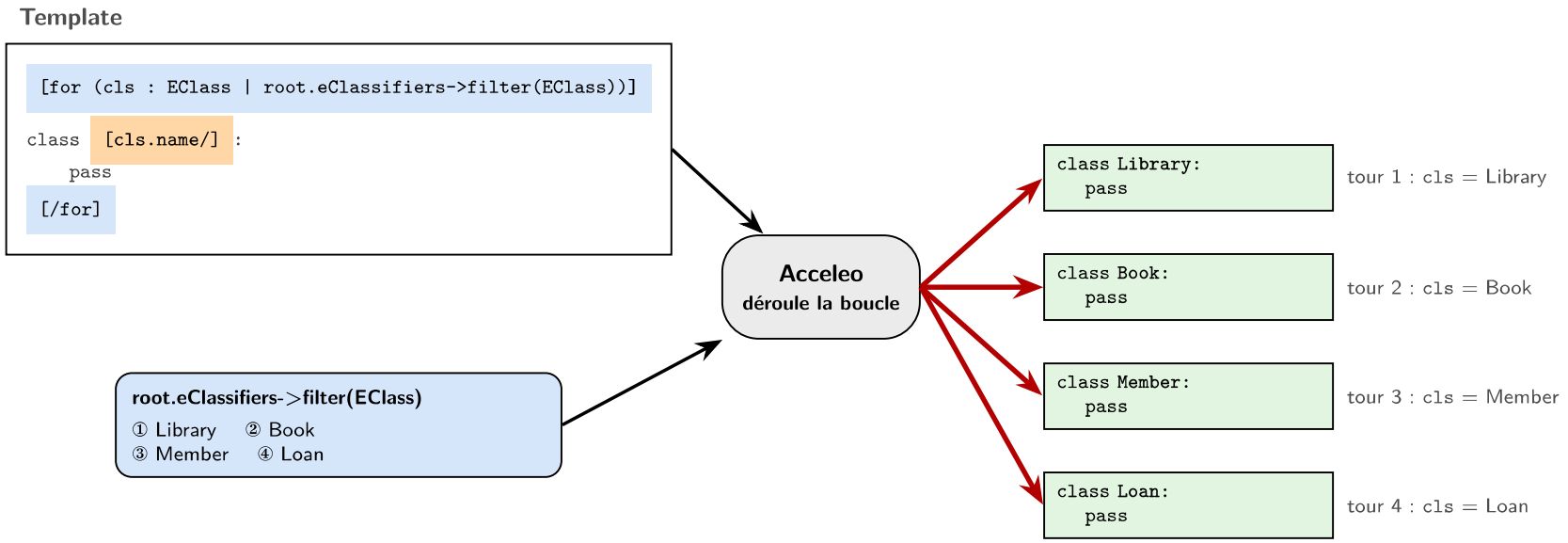
Opération	Exemple	Résultat
`->size()`	`cls.eAttributes->size()`	Book → 5
`->select(x | cond)`	`cls.eAttributes->select(a | a.lowerBound = 1)`	les attributs obligatoires
`->exists(x | cond)`	`cls.eReferences->exists(r | r.many)`	vrai pour Library
`->filter(Type)`	`root.eClassifiers->filter(EClass)`	seulement les EClass
`->first()`, `->isEmpty()`	`cls.eAttributes->first().name`	Book → 'id'
`oclIsKindOf(T)`	`c.oclIsKindOf(EClass)`	vrai si c est une EClass
`toUpperFirst()`	`attr.name.toUpperFirst()`	'title' → 'Title'
`toUpper()`, `concat()`, `+`	`'get' + attr.name.toUpperFirst()`	'getTitle'
`endsWith()`	`attr.name.endsWith('Id')`	vrai pour memberId

! Le **point** . accède à une propriété d'**un** objet ; la **flèche** -> applique une opération à une **collection**.

La boucle [for]

→ Répéter un morceau de texte pour chaque élément

```
[for (variable : Type | collection)]
    ... texte répété (variable change à chaque tour) ...
[/for]
```



*Le corps de la boucle est **recopié une fois par élément** de la collection ; la variable `cls` change à chaque tour.*

[for] avec separator

→ Le séparateur se place entre les éléments, jamais après le dernier

Sans separator

```
[for (a : EAttribute | cls.eAttributes)][a.name/], [/for]
```

→ `id, title, isbn, author, available ,`

virgule en trop à la fin !

Avec separator

```
[for (a : EAttribute | cls.eAttributes)
separator(', ') ][a.name/][/for]
```

→ `id, title, isbn, author, available`

le séparateur est placé **entre** les éléments

Cas du tutoriel : le constructeur Python

```
def __init__(self[for (attr : EAttribute | cls.eAttributes)], [attr.name/]=None[/for]):
```

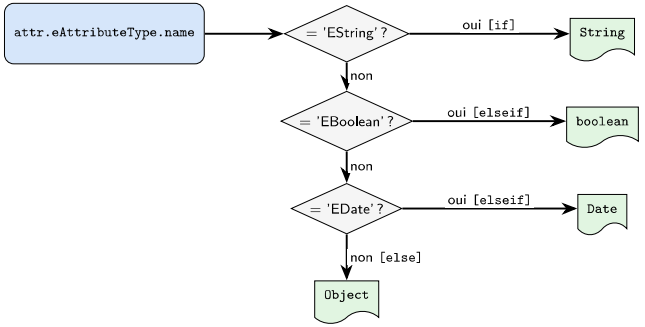
Ici la virgule est **devant** chaque attribut (après `self`) : pas besoin de `separator`. Résultat pour Book :

```
def __init__(self, id=None, title=None, isbn=None, author=None, available=None):
```

La conditionnelle [if]

→ Produire un texte différent selon le modèle

```
[for (attr : EAttribute | cls.eAttributes)]
[if (attr.eAttributeType.name = 'EBoolean')]
    # [attr.name/] est un booléen
[elseif (attr.lowerBound = 1)]
    # [attr.name/] est obligatoire
[else]
    # [attr.name/] est facultatif
[/if]
[/for]
```




```
[if (t = 'EString')]String
[elseif (t = 'EBoolean')]boolean
[elseif (t = 'EDate')]Date
[else]Object
[/if]
```

Résultat pour Member

```
    # memberId est obligatoire
    # name est obligatoire
    # email est facultatif
```

- `[elseif]` et `[else]` sont facultatifs.
- Opérateurs : `=`, `<>`, `<`, `>`, `and`, `or`, `not`.

La variable locale [let]

→ Nommer une valeur pour la réutiliser

```
[let n : Integer = cls.eAttributes->size()]
# [cls.name/] : [n/] attribut[if (n > 1)]s[/if]
[/let]
```

Résultat

```
# Book : 5 attributs
# Library : 1 attribut
```

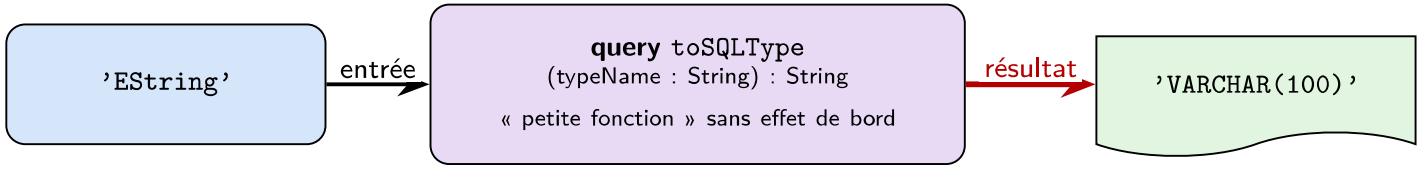
- `[let nom : Type = expression] ... [/let]` : la variable n'existe **qu'entre** ces deux balises.
- Elle est **non modifiable** (pas d'affectation `n = n + 1`).
- Utile pour éviter de recalculer une expression longue.

Correction du tutoriel

En Acceleo 3, `[let]` est un **bloc** : il faut le refermer par `[/let]` (la forme `[let x : Integer = .../]` n'est pas valide).

Les queries : le principe

→ Une petite fonction qui calcule une valeur



Type Ecore	SQL	Java
EString	VARCHAR(100)	String
EInt	INT	int
EBoolean	BOOLEAN	boolean
EDate	DATE	Date
EDouble	DECIMAL(10,2)	double
autre	VARCHAR(255)	Object

*Le tableau de correspondance des types est **la connaissance de la plateforme** (le PDM) : on l'isole dans une query.*

Les queries : l'exemple du tutoriel

→ Convertir un type Ecore en type SQL

```
[query public toSQLType(typeName : String) : String =
    if typeName = 'EString' then 'VARCHAR(100)'
    else if typeName = 'EInt' then 'INT'
    else if typeName = 'EBoolean' then 'BOOLEAN'
    else if typeName = 'EDate' then 'DATE'
    else 'VARCHAR(255)'
    endif endif endif endif
/]
```

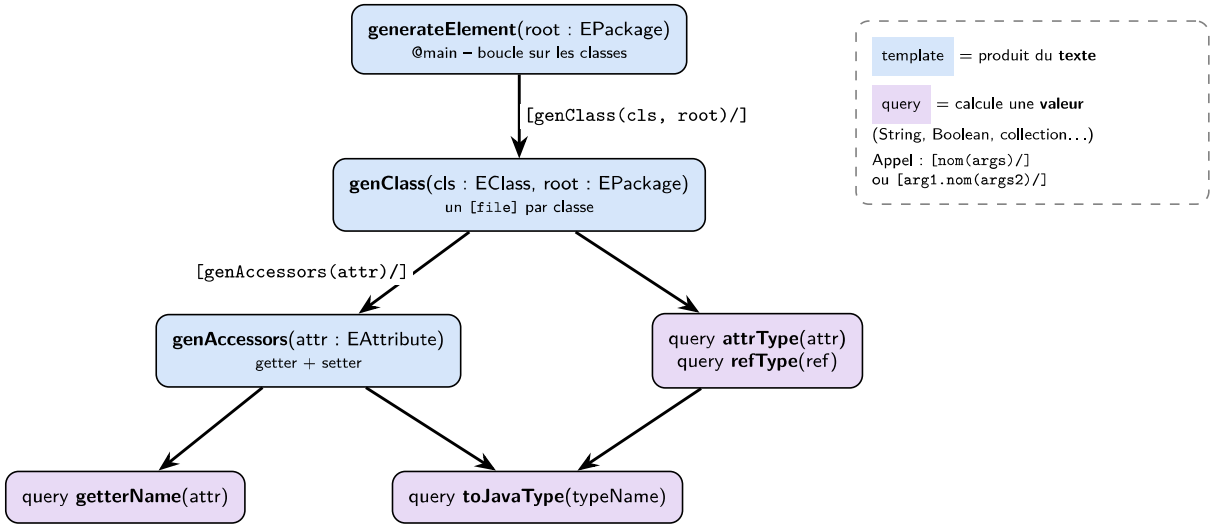
Appel dans le template

```
[attr.name/] [attr.eAttributeType.name.toSQLType
    ()/]
```

- `attr.eAttributeType.name` donne le nom du type Ecore (`'EDate'`)...
- ... puis `.toSQLType()` le convertit : `'EDate'.toSQLType()` → `'DATE'`.
- Syntaxe : `[query public nom(param : Type) : TypeRetour = expression /]`.
- Le `if ... then ... else ... endif` d'OCL est une **expression** : il a toujours un `else` et renvoie une valeur.

Templates et queries : qui fait quoi ?

→ Découper un générateur en morceaux



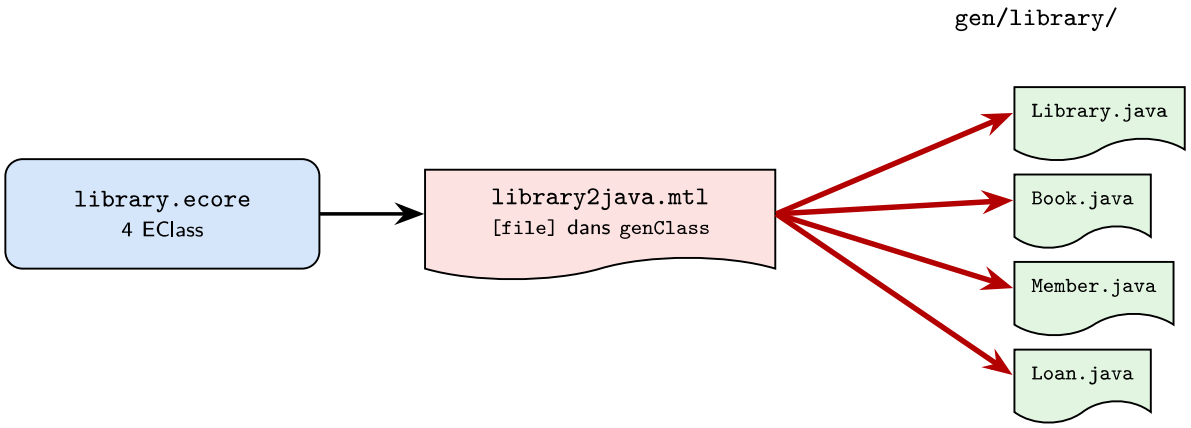
- Un **template** produit du **texte** ; une **query** calcule une **valeur**.
- Un template peut en **appeler** un autre : `[genClass(cls, root)/]`.
- Écriture équivalente : `[cls.genClass(root)/]` (le 1er argument devant le point).
- Un template par « morceau » : plus lisible, plus facile à tester.

Un fichier par élément : [file]

→ Où et comment écrire le résultat

```
[file (root.name + '/' + cls.name + '.java', false, 'UTF-8')]
...
[/file]
```

- 1er argument : le **chemin** du fichier, calculé (ici `library/Book.java`, relatif au dossier cible).
- 2e argument : `false` = écraser ; `true` = ajouter à la fin. 3e argument : l'encodage.
- Un `[file]` **dans une boucle** crée **un fichier par tour** de boucle.




```
public class Book {
  private String id; ...
  public Book() {}
  public Book(String id, ...) {...}
  public String getId() {...}
  public boolean isAvailable() {...}
  public String toString() {...}
}
```

*Un `[file]` placé **dans une boucle** (ou dans un template appelé par une boucle) crée **un fichier par élément**.*

Espaces et retours à la ligne

→ La cause n° 1 des « résultats bizarres »

Template		Résultat
`[for (attr : EAttribute | cls.eAttributes)]`	✗ ligne non produite	`self.id = id`
`  self.[attr.name/] = [attr.name/]`	✓ recopiée (avec ses 4 espaces)	`self.title = title`
`[/for]`	✗ ligne non produite	`self.isbn = isbn`

*Règle : une ligne qui ne contient **qu'une balise de bloc** (`[for]`, `[/for]`, `[if]...`) disparaît du résultat.*
*Tout le reste (espaces, tabulations, retours à la ligne) est recopié **exactement**.*

- Pour une ligne **dans** une boucle, indentez **le texte** (comme dans le fichier final), pas la balise.
- Une ligne vide dans le template = une ligne vide dans le résultat (à chaque tour de boucle !).

Zones protégées : [protected]

→ Garder le code écrit à la main lors d'une régénération

Dans le template

```
// [protected ('methodes ' + cls.name)]
// Ajoutez ici votre code : il sera conservé
// [/protected]
```

Dans le fichier généré

```
// Start of user code methodes Book
// Ajoutez ici votre code : il sera conservé
// End of user code
```

① 1re génération

```
public class Book {
  ...
 // Start of user code methodes Book
 // End of user code
}
```

② le développeur complète

```
public class Book {
  ...
 // Start of user code methodes Book
 public void emprunter() {...}
 // End of user code
}
```

③ modèle modifié + régénération

```
public class Book {
 private int nbPages; (nouveau)
 ...
 // Start of user code methodes Book
 public void emprunter() {...}
 // End of user code
}
```

*Le code écrit **dans** la zone protégée est **conservé** ; tout code écrit **en dehors** est **écrasé** à la régénération.*

Les commentaires

→ Documenter le template sans polluer le résultat

- `[comment texte libre /]` : ignoré par le moteur, n'apparaît **pas** dans le fichier généré.
- `[comment @main/]` : commentaire **spécial** qui marque le template **point d'entrée**.
- `[comment encoding = UTF-8 /]` : 1re ligne du fichier, encodage du `.mtl`.
- Pour un commentaire **dans le code produit**, on l'écrit comme du texte statique : `# ...` (Python), `// ...` (Java), `- ...` (SQL).

> **!** Mettez un `[comment ... /]` au-dessus de chaque template et de chaque query : à quoi sert-il ? que reçoit-il ?

Question flash

?

Écrire l'expression qui donne la **liste des noms des attributs obligatoires** (`lowerBound = 1`) d'une classe `cls`, séparés par des virgules.

Question flash

? Écrire l'expression qui donne la **liste des noms des attributs obligatoires** (`lowerBound = 1`) d'une classe `cls`, séparés par des virgules.

Réponse

`[for (a : EAttribute | cls.eAttributes->select(x | x.lowerBound = 1)) separator(', ')][a.name/][/for]` → pour Member : `memberId, name` (email est facultatif).

Question flash

?

Que produit `[for (r : EReference | cls.eReferences)][r.name/]:[r.eReferenceType.name/] [/for]` pour la classe **Loan** ? Et pour **Book** ?

Question flash

? Que produit `[for (r : EReference | cls.eReferences)][r.name/]:[r.eReferenceType.name/] [/for]` pour la classe **Loan** ? Et pour **Book** ?

Réponse

Pour Loan : `book:Book member:Member ` (avec un espace final !). Pour Book : **rien** (une chaîne vide), car Book n'a aucune référence : la boucle fait zéro tour.

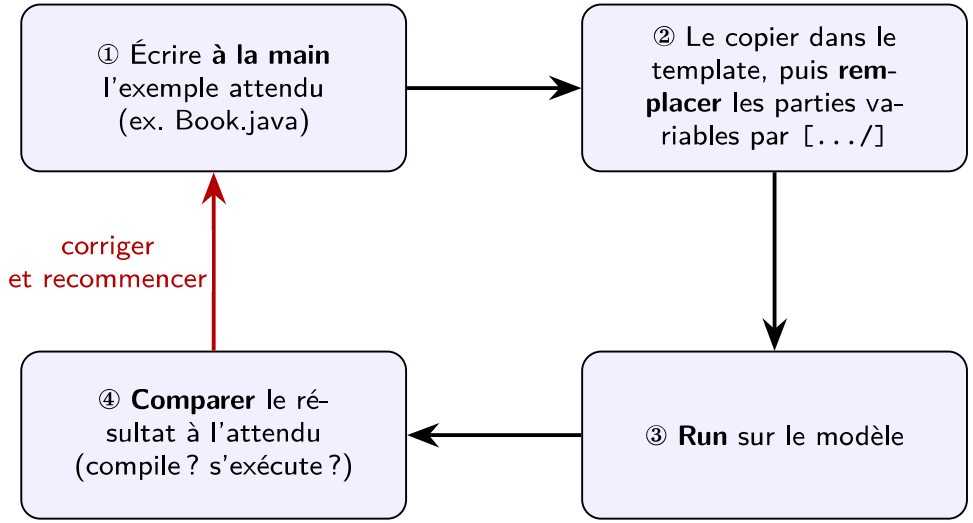
Cas complet : Library.ecore

Comment générer Python, SQL et Java à partir de Library.ecore ?

Le plan de travail

→ Un modèle, trois générateurs (comme dans le tutoriel)

Module	Fichier(s) produit(s)	Nouveautés
`library2python.mtl`	`library_model.py`	boucles imbriquées, query `initValue`
`library2sql.mtl`	`library.sql`	queries de types, clés primaires et étrangères
`library2java.mtl`	`library/Book.java...` (4 fichiers)	un `[file]` par classe, getters/setters, `toString()`, zone protégée



Méthode « exemple d'abord »

On ne part jamais d'une page blanche : un template, c'est **du code exemple dont on a percé les trous.** On avance par petits pas : une boucle, on teste ; une condition, on teste.

Python : le template complet

→ library2python.mtl (amélioré par rapport au tutoriel)


```
[file ('library_model.py', false, 'UTF-8')]
# -*- coding: utf-8 -*-
[for (cls : EClass | root.eClassifiers->filter(EClass))]


class [cls.name/]:
    def __init__(self[for (attr : EAttribute | cls.eAttributes)], [attr.
      name/]=None[/for]):
[for (attr : EAttribute | cls.eAttributes)]
        self.[attr.name/] = [attr.name/]
[/for]
[for (ref : EReference | cls.eReferences)]
        self.[ref.name/] = [ref.initValue()/]
[/for]
[/for]
[/file]
```

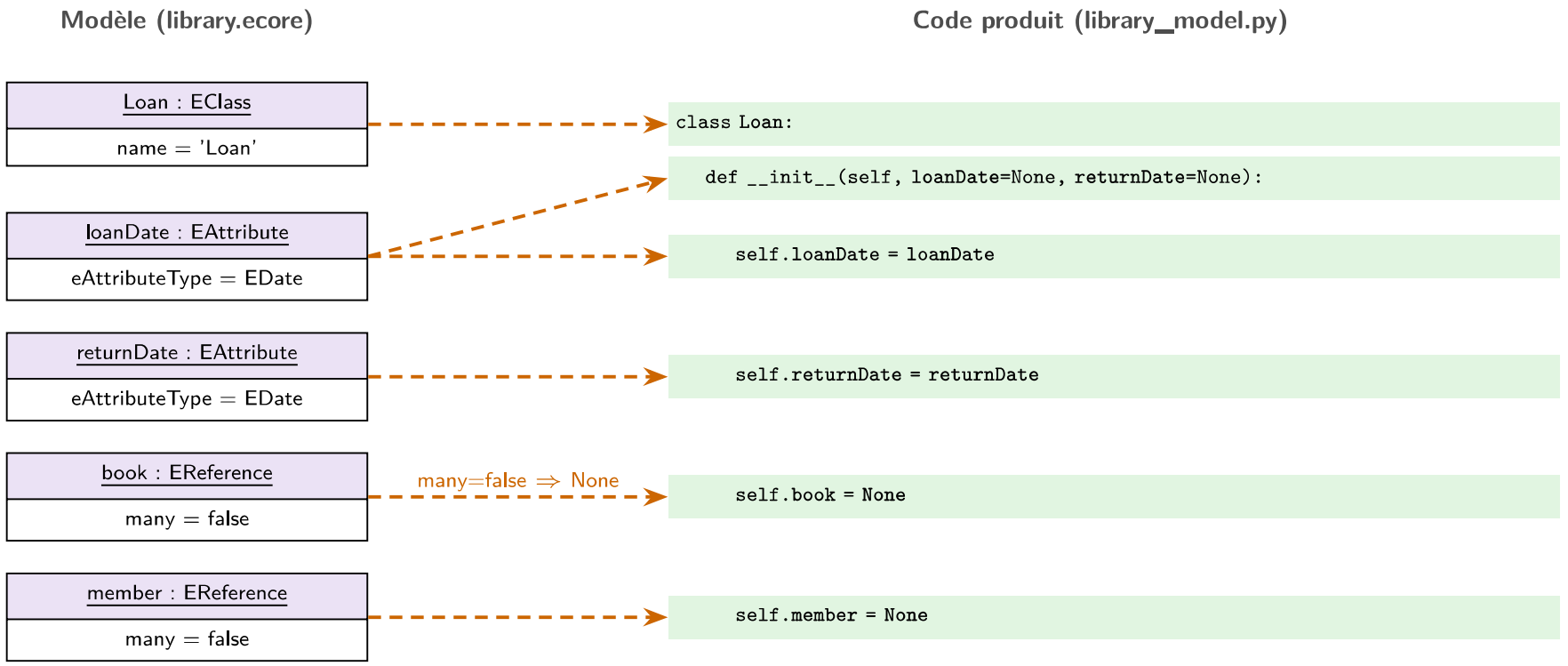
- Boucle **externe** : une classe par EClass.
- Boucles **internes** : paramètres du constructeur, puis une ligne par attribut et par référence.
- **Amélioration** : une référence multiple (`0..*`) devient une liste vide `[]`, une référence simple (`0..1`) devient `None`.

```
[query public initValue(ref : EReference) :
      String =
    if ref.many then '[]' else 'None' endif
/]
```

Python : traçabilité modèle → code

→ D'où vient chaque ligne ?

CIM | PIM | PSM | **Code**



*Chaque élément du modèle produit une (ou plusieurs) lignes : c'est la **traçabilité** modèle → code.*

Python : le résultat et son test

→ Le code généré s'exécute tel quel


```python
class Book:
    def __init__(self, id=None, title=None, isbn=None, author=
      None, available=None):
        self.id = id
        self.title = title
        self.isbn = isbn
        self.author = author
        self.available = available


class Loan:
    def __init__(self, loanDate=None, returnDate=None):
        self.loanDate = loanDate
        self.returnDate = returnDate
        self.book = None
        self.member = None
```

Petit programme de test

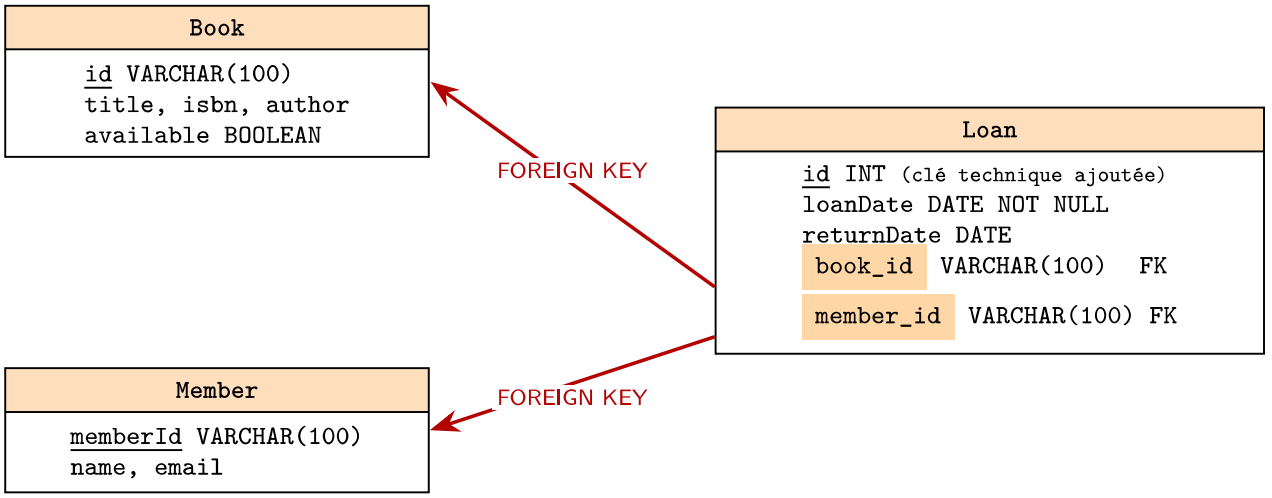
```python
from library_model import Library, Book, Member, Loan

lib = Library("ENSIAS")
b = Book("B1", "Le Petit Prince", "978-2070612758", "Saint-
      Exupéry", True)
m = Member("M1", "Sara", "sara@um5.ac.ma")
lib.book.append(b); lib.member.append(m)
l = Loan("2026-11-02"); l.book = b; l.member = m
print(b)   # Book(id='B1', title='Le Petit Prince', ...)
```

! Le tutoriel produisait `self.book = []` pour Loan ; avec `initValue()` on obtient None, ce qui respecte la multiplicité `0..1`.

SQL : les règles de traduction

→ Exercice 1 du tutoriel, en version complète



Règles appliquées

① 1 EClass → 1 table
② 1 EAttribute → 1 colonne (type via `toSQLType`)
③ `lowerBound = 1` → NOT NULL
④ attribut `id` ou `...Id` → clé primaire, sinon `id INT`
⑤ EReference 0..1 non contenante → colonne `_id` + FK

Avant d'écrire un template, on écrit **les règles** (en français) et **un exemple** attendu. Le template n'est que leur traduction.

SQL : le template

→ library2sql.mtl – le corps de la boucle

```
[for (cls : EClass | root.eClassifiers->filter(EClass))]

CREATE TABLE [cls.name/] (
[if (not cls.hasPK())]
    id INT NOT NULL,
[/if]
[for (attr : EAttribute | cls.eAttributes)]
    [attr.name/] [attr.eAttributeType.name.toSQLType()/][if (attr.lowerBound > 0)] NOT NULL[/if],
[/for]
[for (ref : EReference | cls.eReferences->select(r | not r.containment and not r.many))]
    [ref.name/]_id [ref.eReferenceType.pkType()/],
[/for]
    PRIMARY KEY ([cls.pkName()/])[for (ref : EReference | cls.eReferences->select(r | not r.containment and not r.many))],
    FOREIGN KEY ([ref.name/]_id) REFERENCES [ref.eReferenceType.name/]([ref.eReferenceType.pkName()/])[/for]
);
[/for]
```

- Astuce anti-virgule : la clause `PRIMARY KEY` est **toujours** la dernière ligne ; chaque ligne précédente se termine donc par une virgule, et chaque `FOREIGN KEY` **commence** par une virgule.

SQL : les queries (helpers)

→ Toute la « connaissance SQL » est rangée ici

```
[query public isPK(attr : EAttribute) : Boolean =
    attr.name = 'id' or attr.name.endsWith('Id') /]
[query public hasPK(cls : EClass) : Boolean =
    cls.eAttributes->exists(a | a.isPK()) /]
[query public pkName(cls : EClass) : String =
    if cls.hasPK() then cls.eAttributes->select(a | a.isPK())->first().
      name
    else 'id' endif /]
[query public pkType(cls : EClass) : String =
    if cls.hasPK()
    then cls.eAttributes->select(a | a.isPK())->first().eAttributeType.
      name.toSQLType()
    else 'INT' endif /]
```

- `toSQLType` : le tableau des types (vu plus haut, + `EDouble`).
- `isPK` : **convention** de nommage pour la clé primaire (`id` ou `...Id`).
- `hasPK`, `pkName`, `pkType` : utilisées pour la clé de la table **et** pour les clés étrangères qui la référencent.
- Une classe sans identifiant (Library, Loan) reçoit une colonne technique `id INT`.

SQL : le résultat

→ Extrait de library.sql (exécutable dans SQLite, MySQL, PostgreSQL)

```sql
CREATE TABLE Member (
    memberId VARCHAR(100) NOT NULL,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100),
    PRIMARY KEY (memberId)
);

CREATE TABLE Loan (
    id INT NOT NULL,
    loanDate DATE NOT NULL,
    returnDate DATE,
    book_id VARCHAR(100),
    member_id VARCHAR(100),
    PRIMARY KEY (id),
    FOREIGN KEY (book_id) REFERENCES Book(id),
    FOREIGN KEY (member_id) REFERENCES Member(memberId)
);
```

- Les tables sont créées **dans l'ordre du modèle** : Book et Member existent avant Loan, les clés étrangères sont donc valides.
- `email` et `returnDate` sont `[0..1]` : pas de `NOT NULL`.
- Les **compositions** (Library ♦ Book...) ne sont pas traduites ici : c'est une extension proposée en TP.

Java : un fichier par classe

→ library2java.mtl – le template principal appelle genClass


```
[template public generateElement(root : EPackage)]
[comment @main/]
[for (cls : EClass | root.eClassifiers->filter(EClass))]
[genClass(cls, root)/]
[/for]
[/template]

[template public genClass(cls : EClass, root : EPackage)]
[file (root.name + '/' + cls.name + '.java', false, 'UTF-8')]
public class [cls.name/] {
}
[/file]
[/template]
```

- Le template `@main` ne fait qu'une **boucle** sur les classes.
- Pour chaque classe, il **appelle** `genClass(cls, root)`.
- `genClass` contient le `[file]` : **un fichier par classe**, rangé dans le dossier du paquetage (`library/`).
- On passe `root` en paramètre pour connaître le nom du paquetage (`package library;`).

! Attributs privés, constructeur, getters/setters et `toString()` pour Library, Book, Member et Loan.

Java : genClass (1/2)

→ En-tête, attributs et constructeur vide


```
[template public genClass(cls : EClass, root : EPackage)]
[file (root.name + '/' + cls.name + '.java', false, 'UTF-8')]
package [root.name/];

import java.util.ArrayList;
import java.util.Date;
import java.util.List;

public class [cls.name/] {

[for (attr : EAttribute | cls.eAttributes)]
    private [attr.attrType()/] [attr.name/];
[/for]
[for (ref : EReference | cls.eReferences)]
    private [ref.refType()/] [ref.name/][if (ref.many)] = new ArrayList
      <>()[/if];
[/for]

    public [cls.name/]() {
    }
```

- `attrType()` et `refType()` : queries qui traduisent les types (diapositive suivante).
- Une référence **multiple** est initialisée : `= new ArrayList<>()`.
- Le texte Java (accolades, `private`, `;`) est **statique** : il est recopié tel quel.

Java : genClass (2/2) et genAccessors

→ Constructeur complet, accesseurs, toString()


```
    public [cls.name/]([for (attr : EAttribute | cls.eAttributes)
      separator(', ')][attr.attrType()/] [attr.name/][/for]) {
[for (attr : EAttribute | cls.eAttributes)]
        this.[attr.name/] = [attr.name/];
[/for]
    }
[for (attr : EAttribute | cls.eAttributes)]
[genAccessors(attr)/]
[/for]
```

```
    @Override
    public String toString() {
        return "[cls.name/]{" + [for (attr : EAttribute | cls.
      eAttributes) separator(' + ", " + ')]"[attr.name/]=" + [attr.
      name/][/for] + "}";
    }
```

```
[template public genAccessors(attr : EAttribute)]

    public [attr.attrType()/] [attr.getterName()/]() {
        return this.[attr.name/];
    }

    public void set[attr.name.toUpperFirst()/]([attr.
      attrType()/] [attr.name/]) {
        this.[attr.name/] = [attr.name/];
    }
[/template]
```

- Convention JavaBeans : getter `isXxx()` pour un **booléen** (`isAvailable`), `getXxx()` sinon.

Java : les queries de types

→ Le « dictionnaire » Ecore → Java

```
[query public toJavaType(typeName : String) : String =
    if typeName = 'EString' then 'String'
    else if typeName = 'EInt' then 'int'
    else if typeName = 'EBoolean' then 'boolean'
    else if typeName = 'EDouble' then 'double'
    else if typeName = 'EDate' then 'Date'
    else 'Object' endif endif endif endif endif /]
[query public attrType(attr : EAttribute) : String =
    attr.eAttributeType.name.toJavaType() /]
[query public refType(ref : EReference) : String =
    if ref.many then 'List<' + ref.eReferenceType.name + '>'
    else ref.eReferenceType.name endif /]
[query public getterName(attr : EAttribute) : String =
    (if attr.eAttributeType.name = 'EBoolean' then 'is' else 'get' endif)
    + attr.name.toUpperFirst() /]
```

Ecore	Java
EString	String
EInt	int
EBoolean	boolean
EDouble	double
EDate	Date
référence 0..1	Book
référence 0..*	List<Book>

Java : le résultat

→ gen/library/Book.java (extrait) – compile avec javac


```java
public class Book {

    private String id;
    private String title;
    ...
    private boolean available;

    public Book() {
    }

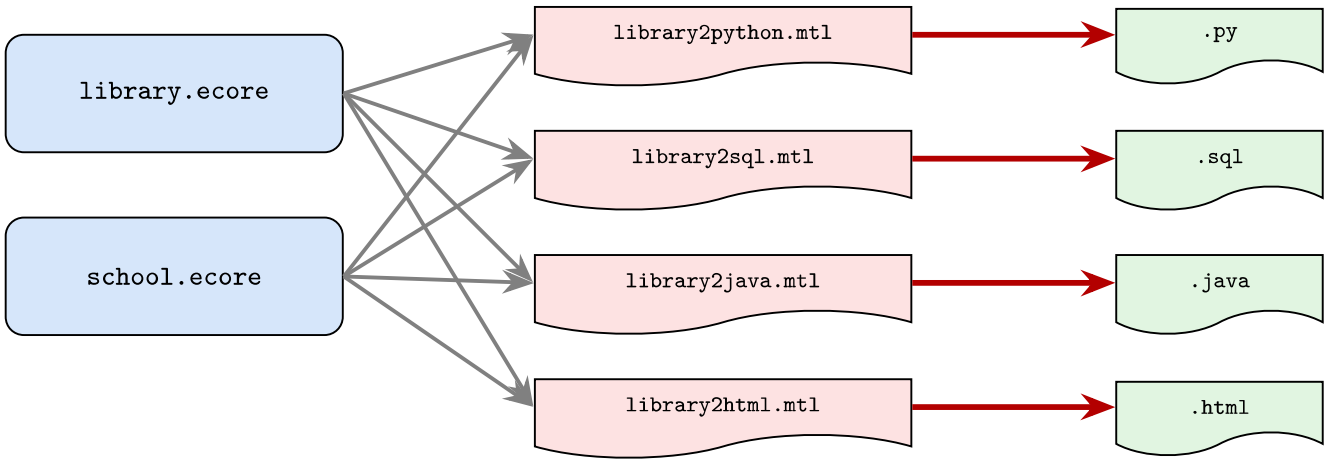
    public Book(String id, String title, String isbn, String
      author, boolean available) {
        this.id = id;
        ...
    }

    public String getId() {
        return this.id;
    }
    ...
    public boolean isAvailable() {
        return this.available;
    }
```

- 4 fichiers dans `gen/library/` : `Library.java`, `Book.java`, `Member.java`, `Loan.java`.
- Library contient `List<Book> book = new ArrayList<>()` (référence multiple).
- Loan contient `Book book` et `Member member` avec getter **et** setter.
- En fin de classe : la **zone protégée** pour le code métier écrit à la main.

Un modèle, plusieurs cibles... et plusieurs modèles

→ La vraie puissance de l'approche



1 modèle → N cibles :
on ajoute un template par plateforme.

N modèles → 1 template :
le template est écrit pour **Ecore**, il marche pour tout `.ecore`.

- Les 4 templates fonctionnent **sans modification** sur `school.ecore` (Student, Course, Enrollment) : à tester en TP.
- Ajouter une cible (TypeScript, C#...) = écrire **un template de plus**, pas réécrire le code.

Question flash

?

Dans `school.ecore`, `Enrollment` a un attribut `grade : EDouble` et deux références **obligatoires** (`1..1`) vers Student et Course. Que faut-il changer dans `library2sql.mtl` pour obtenir `grade DECIMAL(10,2)` et `student_id ... NOT NULL` ?

Question flash

? Dans `school.ecore`, `Enrollment` a un attribut `grade : EDouble` et deux références **obligatoires** (`1..1`) vers Student et Course. Que faut-il changer dans `library2sql.mtl` pour obtenir `grade DECIMAL(10,2)` et `student_id ... NOT NULL` ?

Réponse

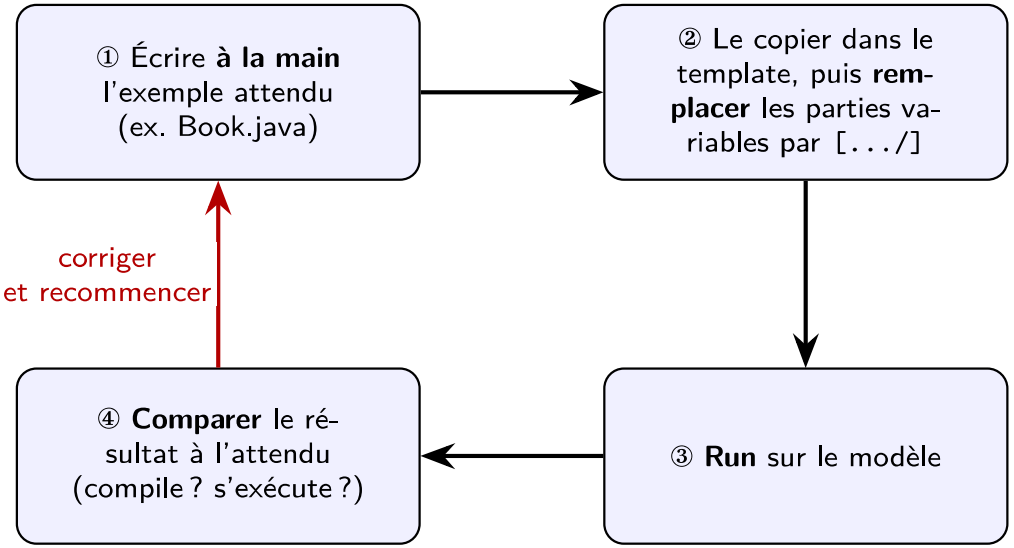
① Ajouter le cas `EDouble` dans la query `toSQLType` (`else if typeName = 'EDouble' then 'DECIMAL(10,2)'`). ② Dans la boucle des colonnes de clés étrangères, ajouter `[if (ref.lowerBound > 0)] NOT NULL[/if]` après le type.

Bonnes pratiques et synthèse

Comment écrire un bon template ? Que retenir ?

Méthode : « l'exemple d'abord »

→ Comment écrire un template sans se perdre



Méthode « exemple d'abord »

On ne part jamais d'une page blanche : un template, c'est **du code exemple dont on a percé les trous.** On avance par petits pas : une boucle, on teste ; une condition, on teste.

Les erreurs les plus fréquentes

→ Et comment les reconnaître

✗ 4 erreurs

```
class [cls.name]:
[for (a | cls.eAttributes)]
 self.[a.name/] = None
[if (a.lowerBound = 1)] # requis
[file ('out.py', false)]
```

✓ corrigé

```
class [cls.name / ]:
[for (a : EAttribute | cls.eAttributes)]
 self.[a.name/] = None
[if (a.lowerBound = 1)] # requis [/if]
[file ('out.py', false, 'UTF-8' )] ... [/file]
```

Symptôme	Cause probable
Erreur « Invalid expression » sur une ligne	`/]` oublié en fin d'expression, ou parenthèse non fermée
Rien n'est généré	pas de `[comment @main/]`, mauvais modèle dans la « Run configuration », ou `[file]` absent
`Unknown type EClass`	URI du module incorrecte (doit être celle d'Ecore)
Virgule en trop, lignes vides en trop	`separator` oublié ; ligne vide ou indentation dans une boucle
Le code ajouté à la main disparaît	écrit **hors** d'une zone `[protected]`

Bonnes pratiques

→ Pour des générateurs lisibles et maintenables

Petits templates

Un template par morceau de fichier (classe, attribut, accesseurs) ; un module par cible.

Queries pour la plateforme

Types, conventions de nommage, clés : tout ce qui dépend de Java/SQL va dans des **queries**.

Code généré = jetable

On ne modifie **jamais** un fichier généré, sauf dans les zones protégées. On corrige le **template** ou le **modèle**.

Tester le résultat

Le code généré doit **compiler** (javac), **s'exécuter** (python) ou **se charger** (SQL). Gardez un fichier « attendu ».

Et Acceleo 4 ?

→ Même idée, syntaxe légèrement différente (AQL)

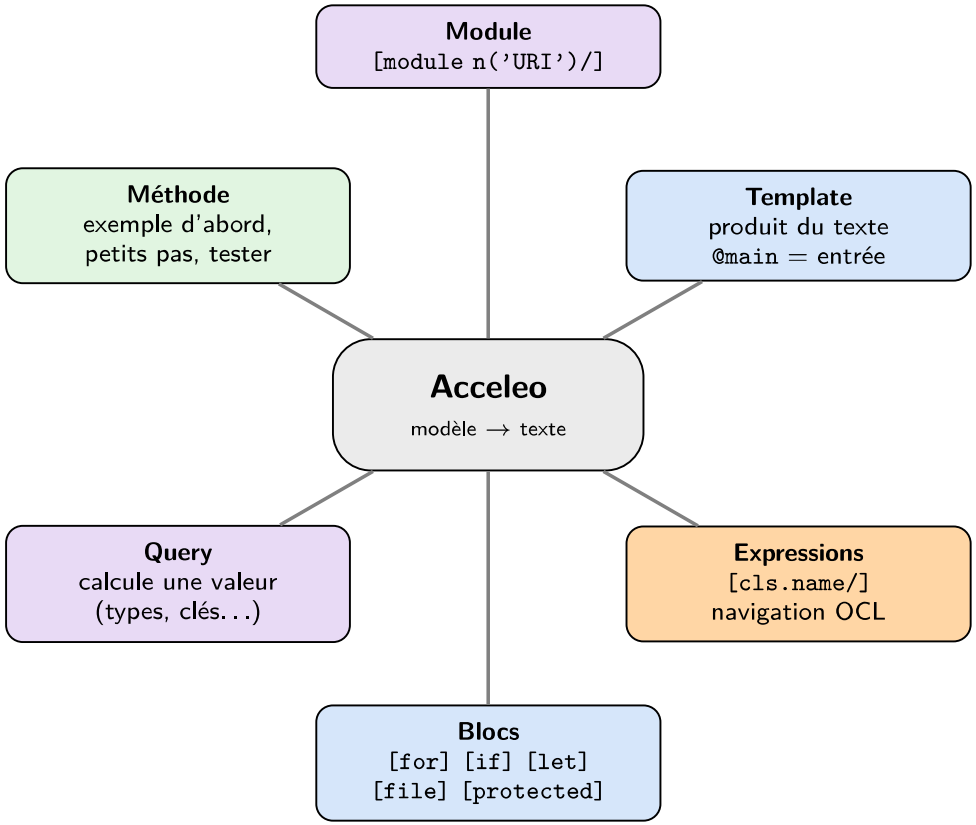
```
[module library2python('http://www.eclipse.org/emf/2002/Ecore')/]

[template public generateElement(root : ecore::EPackage)]
[comment @main/]
[file ('library_model.py', overwrite, 'UTF-8')]
[for (cls : ecore::EClass | root.eClassifiers->filter(ecore::EClass)
      )]
class [cls.name/]:
    pass
[/for]
[/file]
[/template]
```

- Types **qualifiés** : `ecore::EClass`, `ecore::EPackage`.
- `[file (..., overwrite, ...)]` au lieu de `false` (aussi : `append`, `create`).
- Expressions en **AQL** (très proche d'OCL) ; compteur de boucle `clsIndex`.
- Le corps des blocs doit être **indenté de 2 espaces** (non générés).
- Eclipse propose une **migration** automatique Acceleo 3 → 4.

Synthèse

→ Ce qu'il faut retenir



- **M2T** : un modèle → du texte ; Acceleo est le standard Eclipse/OMG.
- Un **template** = du texte final avec des trous `[expression/]`.
- On **navigue** dans le modèle en OCL (`.` et `->`).
- **Blocs** : `[for]`, `[if]`, `[let]`, `[file]`, `[protected]` ; **queries** pour calculer.
- Un même template sert pour **tous** les modèles conformes au méta-modèle.

Aide-mémoire Acceleo 3

→ À garder sous les yeux pendant le TD et le TP

Instruction	Syntaxe
Module	`[module nom('http://www.eclipse.org/emf/2002/Ecore')/]`
Template (point d'entrée)	`[template public gen(root : EPackage)] [comment @main/] ... [/template]`
Fichier	`[file (chemin, false, 'UTF-8')] ... [/file]`
Expression	`[cls.name/], [attr.name.toUpperFirst()/]`
Boucle	`[for (a : EAttribute | cls.eAttributes) separator(', ')] ... [/for]`
Condition	`[if (cond)] ... [elseif (cond)] ... [else] ... [/if]`
Variable	`[let n : Integer = cls.eAttributes->size()] ... [/let]`
Query	`[query public f(x : T) : R = expression /]`
Appel	`[genClass(cls)/]` ou `[cls.genClass()/]`
Zone protégée	`// [protected ('id')] ... // [/protected]`
Commentaire	`[comment texte /]`

Pour s'entraîner

→ TD et TP associés

- **TD** (papier) : prédire le résultat d'un template, écrire des expressions OCL, compléter et corriger des templates (Python, SQL, Java, HTML).
- **TP** (Eclipse) : suivre le tutoriel complet `Library.ecore` → Python, SQL, Java ; puis extensions : HTML, `school.ecore`, zones protégées.
- Ressources fournies : `library.ecore`, `school.ecore`, squelettes `.mtl` à compléter, fichiers **attendus**.

> **!** Combiner ATL (M2M : PIM → PSM) et Acceleo (M2T : PSM → code) : c'est la chaîne MDA complète de la Lecture 2.

Question de synthèse

→ Question flash

Question flash

?

Votre équipe veut ajouter au modèle un attribut `nbPages : EInt` à Book. Citez **tout** ce qui doit être modifié pour que le Python, le SQL et le Java soient à jour.

Question flash

?

Votre équipe veut ajouter au modèle un attribut `nbPages : EInt` à Book. Citez **tout** ce qui doit être modifié pour que le Python, le SQL et le Java soient à jour.

Réponse

Seulement le modèle `library.ecore` (ajout de l'attribut), puis relancer les 3 générateurs. Les templates gèrent déjà `EInt` (`int` en Java, `INT` en SQL). Le code métier écrit dans les **zones protégées** est conservé.